Case Study: Reasoning Gym

Endless, verifiable reasoning tasks for RL training and evaluation

Stojanovski et al. · NeurIPS 2025 Datasets and Benchmarks, spotlight

Learning outcomes

After this session, you can:

1. Explain why generators beat fixed datasets for RLVR.
2. Generate tasks and score answers with Reasoning Gym.
3. Use the difficulty dial, curricula and task mixes.
4. Read the paper’s evaluation and training results.
5. Name the limits of procedural reasoning tasks.
6. Write a new task of your own, for example an Indic one.

The problem with fixed datasets

Cost	Human-written question–answer pairs are expensive.
Quality	Data scraped from the internet is not reliable.
Memorization	A model can learn the answers instead of the method.
Saturation	A fixed benchmark stops showing progress once models master it.

Do not store tasks. Generate them, and check each answer with code.

More than 100 generators, each with its own verifier and difficulty settings.

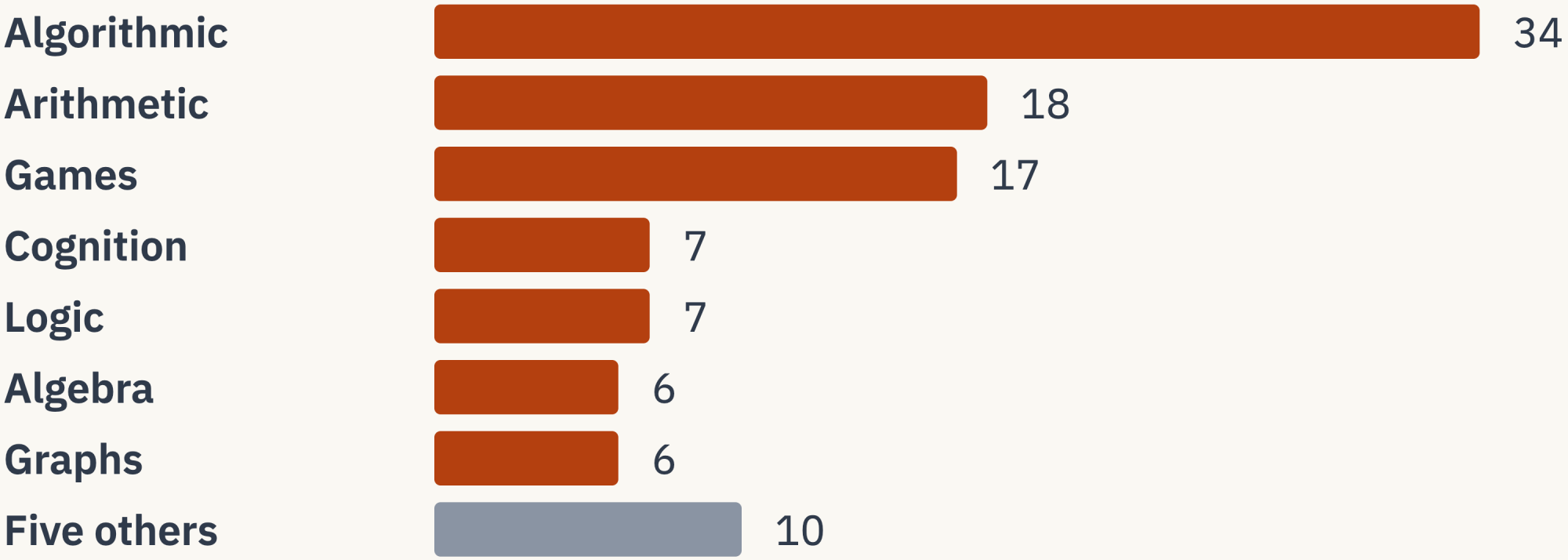
Three design choices

Code checks answers	Every task has an automatic verifier. No human judgment.
Large solution spaces	Many different tasks per generator, so the model must learn a method, not an answer.
Tunable settings	Difficulty, structure and style can each change on their own.

Reasoning Gym as an RL environment

Part	In Reasoning Gym
Task set	A generator plus a config plus a seed
Harness	None: one prompt, one answer
State	None
Grader	score_answer(answer, entry), from 0 to 1
Reset	A new seed gives a new task

105 tasks in 12 groups



Five others: ARC 3, geometry 2, code 2, induction 2, probability 1. Counted in version 0.1.19.

Example 1 · Arithmetic: leg_counting

Your task is to count how many legs there are in total when given a list of animals.

Now, how many legs are there in total if you have 3 sea slugs, 12 deers, 2 giraffes, 11 elephants?

Answer: 100 **Score:** 1.0 if right, 0.0 if wrong

Example 2 · Logic: knights_knaves

An island has only sages and fools. Sages always tell the truth. Fools always lie. You meet Zoey and Riley.

Zoey says: “Riley is a fool.” Riley says: “Zoey is a sage or Riley is a sage.”

Who is a sage, and who is a fool?

Answer: Zoey is a fool, and Riley is a sage.

Example 3 · Games: maze

```
> > > > > > > > > >
> e e e e > e > >
> e e > > > > > >
> e e e e e e > >
> e > e e > > e >
> > e z > 3 e > >
> e e e > e > e >
> e e e e e > e >
> > > > > > > > > >
```

Go from **3** (start) to **z** (goal).

“>” is a wall. “e” is a passage.

What is the minimum number of steps?

Answer: 6

Five lines to make tasks and score them

```
import reasoning_gym as rg

data = rg.create_dataset("leg_counting", size=10, seed=42)
x = data[0]
print(x["question"], x["answer"])
score = data.score_answer(answer=reply, entry=x)
```

Install: pip install reasoning-gym. The same seed always gives the same tasks.

What the grader rewards

Reply (answer is 100)	Score
"100"	1.0
" 100"	0.75
"100 legs"	0.375
"The answer is 100"	0.18
"101"	0.0

Extra text lowers the score, even a space.

Some tasks accept many right answers. Knights and knaves accepts either order of names.

Extract the final answer before you score it.

The difficulty dial

Easy

`max_animals=2`
`max_instances=3`

“3 dogs, 1 wasp”

Hard

`max_animals=20, max_instances=50`

“36 dogs, 28 wasps, 4 flatworms, 37 leeches, 8 birds, 15 beetles, 41 sheep, 41 sea slugs, 38 tigers, …”
(17 kinds)

Same task, same code. Only the settings change.

Curricula: step-by-step difficulty

```
from reasoning_gym.factory import create_curriculum
c = create_curriculum("leg_counting")
c.generate_configuration(...)
# min_animals=1, max_animals=1
c.increment_attr_level("num_animals")
c.generate_configuration(...)
# min_animals=1, max_animals=2
```

102 of the 105 tasks ship a curriculum. The paper reports that adaptive difficulty beats fixed difficulty.

Mix tasks with weights

```
from reasoning_gym.composite import DatasetSpec
specs = [DatasetSpec("leg_counting", weight=2, config={}),
         DatasetSpec("basic_arithmetic", weight=1, config={})]
mix = rg.create_dataset("composite", size=6, seed=1,
                        datasets=specs)
```

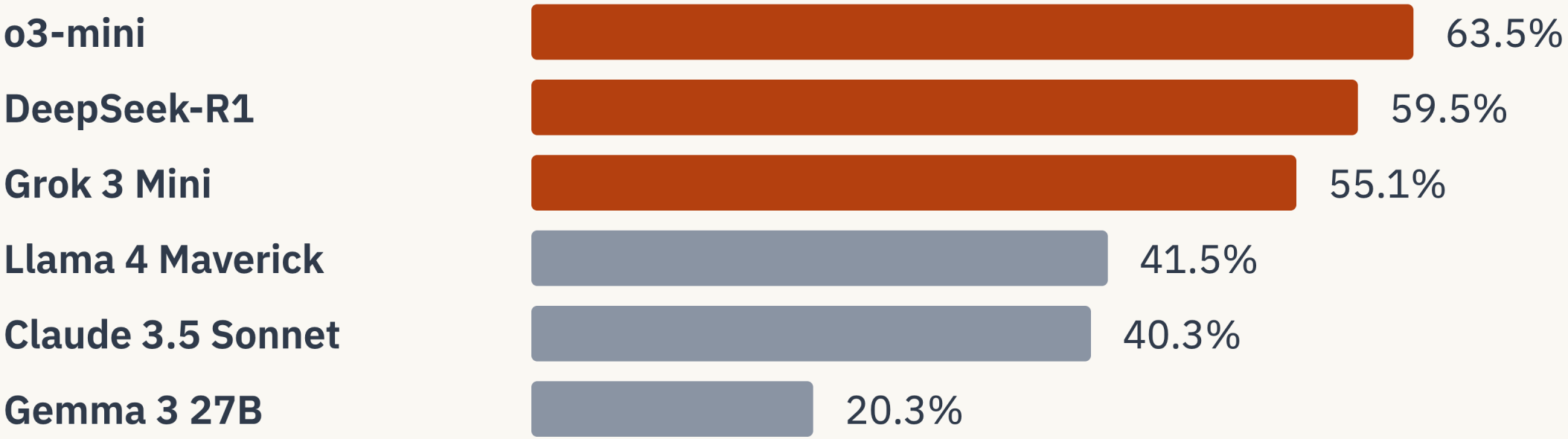
Result: 4 leg_counting tasks and 2 basic_arithmetic tasks. Each task records its source, so the right grader scores it.

Part 2

As a benchmark

Zero-shot scores of frontier models on hard settings

Reasoning models lead by 22 points



Orange: reasoning models. **Grey:** general models. Average over all tasks, hard settings, zero-shot.

The difficulty cliff: easy to hard

Category	o3-mini	DeepSeek-R1
Code	−71.9	−61.8
Graphs	−33.8	−29.6
Geometry	−33.1	−11.8
Algorithms	−25.6	−27.9

Drop in score, in percentage points, when the same tasks get harder settings.

Part 3

For RLVR training

GRPO on a 3B model: what improved, and what transferred

The training setup

Model	Qwen2.5-3B-Instruct
Algorithm	GRPO, with verl as the trainer
Reward	Correct answer: 1.0. Right <think> and <answer> format: 0.2.
Compute	About 1,500 A6000 GPU hours for all experiments. Three runs each.

Train and test in the same category

Category	Before	After	Change
Algebra	5.0	16.7	+11.7
Algorithmic	52.3	59.7	+7.4
Arithmetic	89.7	96.0	+6.3
Cognition	40.3	42.3	+2.0
Games	0.0	3.3	+3.3

Accuracy (Acc@3, %) of Qwen2.5-3B before and after GRPO on that category.

Skills transfer across categories

Trained on	Tested on	Change
Algorithmic	Algebra	+29.1
Algorithmic	Geometry	+22.3
Games	Algebra	+21.8
Algebra	Arithmetic	+16.6
Logic	Cognition	+13.3
Algebra	ARC	−2.3

Acc@3 change in points, Qwen2.5-3B. From the paper’s Table 2.

Puzzles help on real benchmarks

+9.7

MATH

48.5 → 58.2

+7.7

Big-Bench Hard

8.7 → 16.3

+0.5

GSM8K

76.2 → 76.7

Qwen2.5-3B trained with GRPO on the RG-Math mix. GSM8K barely moved.

Limits to keep in mind

Templates	Models may learn solution templates, not robust strategies.
Spatial tasks	Visual puzzles written as text stay hard. Games training plateaued early.
Saturation	Basic arithmetic is close to solved. Hard algorithms are not.
Realism	Puzzles are not real work. They have one turn and no tools.

What is in the GitHub repository

Folder	What it gives you
examples/trl	GRPO with TRL
examples/unsloth	LoRA GRPO on one GPU
examples/verifiers	Prime Intellect’s verifiers
training/	The paper’s verl runs and configs
eval/	Score any API model
GALLERY.md	Every task, with examples and settings

Part 4

Try it, then build your own

A no-GPU exercise, a custom task, and Indic ideas

In-class exercise: find the learning zone

1. Pick one task. Make 50 easy tasks and 50 hard tasks.
2. Ask a small model for each answer, inside answer tags.
3. Extract the answer. Score it with score_answer.
4. Compare the pass rates for easy and hard settings.
5. Find settings with a 20% to 60% pass rate.

That setting is where GRPO training would learn the most.

Write your own task: a GST invoice

```
class GstInvoice(ProceduralDataset):
    def __getitem__(self, idx):
        rng = random.Random(self.seed + idx)
        price, qty = rng.randint(50, 900), rng.randint(1, 5)
        rate = rng.choice([5, 18])
        total = round(price * qty * (1 + rate / 100), 2)
        q = f"{qty} x Rs {price}, GST {rate}%. Total kitna?"
        return {"question": q, "metadata": {},
                "answer": f"{total:.2f}"}

rg.register_dataset("gst_invoice", GstInvoice, GstConfig)
```

Indic task ideas for Reasoning Gym

Lakh and crore	“2.5 lakh + 80 hazaar = ?” Convert and add Indian number words.
Kinship puzzles	Family puzzles with Hindi terms such as mausi, chacha and bua.
Train routes	Shortest connection on a generated railway timetable.
Script tasks	Convert between Devanagari and Roman script with one fixed scheme.

Questions for discussion

1. Can a model hack the partial-credit grader? How?
2. Why might algorithm practice help algebra, but not ARC?
3. Is learning a solution template a bad thing?
4. What real skill could your own generator teach?

Read the originals

- Paper: Reasoning Gym (NeurIPS 2025 spotlight)
- GitHub repository and its task gallery
- Training examples: TRL, Unsloth, veRL, verifiers
- Shared evaluation results
- PyPI: pip install reasoning-gym